

HOPCo Senior QA Engineer Task

You will be provided with login credentials for:

1. The myrecovery mobile app (available via App Store or Google Play)

We encourage the use of generative AI tools as part of this exercise. We want to see how you use them as part of your workflow. Please include clear comments specifying what tools were used, where in your process, and any example prompts applied.

For any questions, please email henna@myrecovery.ai. Please aim to send it over by 17th July.

Expected tech: WebdriverIO + TypeScript, Appium (3.x with xcuitest/uiautomator2)
Either platform is fine. You do not need to cover both iOS and Android.

Task 1: Test Design — Acceptance Criteria for the Check-In Flow

Estimated Time: 45 minutes

Objective:

Before you automate anything, define what “correct” looks like for the flow in Task 2, particularly the two ambiguous points called out below.

Instructions:

1. Write Given/When/Then acceptance criteria for
 - (a) dismissing pending surveys on login, where the number of surveys shown is not fixed. It can be zero, one, or several depending on account state and
 - (b) the pain score / daily check-in being correctly reflected on the Progress tab, including which date it's attributed to.
 - Note any product questions or ambiguities you hit while writing these. For example, what should happen if a survey fails to dismiss, or if a check-in already exists for today.
2. Derive a short list of test cases from your acceptance criteria — enough to cover a positive and at least one negative/edge case for each of the two areas above — and note which of these you'd automate in Task 2 versus leave to manual/exploratory testing, and why.

Deliverable:

Written acceptance criteria in BDD format, a short list of derived test cases with your automate-vs-manual call, and a short list of open questions or assumptions. Include this as a markdown file (e.g. TASK1.md) in the same repo or zip you submit for Task 2, rather than as a separate document. We want everything in one place.

Task 2: Automation — Core Regression Flow

Estimated Time: 2 hours

Objective:

Automate the flow below end-to-end for one platform (iOS or Android, your choice), using WebdriverIO + TypeScript + Appium 3.x.

Expected workflow

1. Login as a patient
2. Close all the surveys
3. Go to homepage
4. Go to pain score/ daily check in
5. Complete the score
6. Go to more and click on log out, confirm log out

More advanced

1. Login as a patient
2. Close all the survey
3. Go to homepage
4. Go to pain score/ daily check in
5. Complete the score
6. Go to progress tab
7. Verify Surveys & assessments card - Pain score recorded as 1
8. Go to more and click on log out, confirm log out

Thing to include

- Page Object Model
- A README explaining setup, how to run, assumptions made, and known limitations
- Explicit waits (no hardcoded sleeps as the primary wait strategy)
- Sensible locator strategy
- Error handling awareness(If a step fails, does the suite fail clearly, or hang?)

Deliverable:

Your code implementing the advanced flow, with a README. A link to a git repository is preferred; see Deliverable Format below for details and the zip fallback.

Deliverable Format

- A link to a git repository is preferred (GitHub, GitLab, etc.) either public, or private with henna@myrecovery.ai added as a collaborator so we are not left waiting on access.
- If you would rather not share access to an online repo, a zipped export that includes your .git folder (so commit history is preserved) works too.
- Your README must state the exact environment you used (Node version, Appium version, iOS/Android SDK version, device or emulator/OS version) and the exact command(s) to run the suite.
- Because mobile tooling setup varies a lot machine-to-machine, also include proof that your suite runs: either a short screen recording of a passing run, or the raw test output/report (terminal log or an HTML report such as Allure or Mochawesome) committed to the repo. If we can't get your suite running locally within a few minutes, we'll evaluate from this proof rather than marking it non-functional.
- If you would rather narrate a couple of your trickier design decisions while recording the passing run, that's welcome but not required.